KuduDash Sprint Retrospective Reports

This report brings together the retrospective summaries from all four sprints of the KuduDash project. At the end of each sprint, the team met to discuss how the sprint went and what could be done better. Each retrospective looked at three main things: what went well, what did not go as planned, and what actions the team would take in the next sprint to improve. The report covers everything from the early stages of setting up the project in Sprint 1, all the way through to the final features delivered in Sprint 4. It is meant to give an honest overview of the team's work, challenges, and growth throughout the project.

Sprint Retrospective meeting 1:

In Sprint 1, the team set up the project from scratch, building the backend, designing and implementing the database schemas using Mongoose. The team successfully set up and configured the full MERN stack environment which gave everyone a stable and organised base to build on going forward. The backend REST APIs were built using Node.js and Express. Auth0 Google authentication was integrated to handle secure login for users. The student and vendor dashboards were linked to the backend so they could display live data.

Despite the technical progress, the sprint was not without its problems. Task ownership was not clearly defined at the start which made integration work slower than it needed to be. There were also no regular check-ins during the sprint, which meant that faults were only discovered late, by which point they had already caused delays. Breaking larger tasks into into smaller ones would have made it easier to track progress throughout the sprint rather than only seeing how things were near the deadline. Starting tasks earlier would also have reduced the last minute pressure that some members experienced.

Going into Sprint 2, the team agreed on the following steps based on what came out of the Sprint 1 meetings. All dashboard pages needed to be fully connected to their backend endpoints, with proper error handling and loading states across all fetch calls. Login implementation had to be completed, and the full authentication flow tested end to end. The remaining APIs also needed to be finished so that backend and frontend components could be integrated correctly. Task ownership needed to be clearer from the start, as several tasks had been reassigned mid-sprint due to members not completing what they were responsible for. The team also needed to address participation concerns that had been raised more than once during the sprint without improvement.

Sprint Retrospective meeting 2:

Sprint 2 was where the team made a lot of progress on the core features of the platform. The full cart system was built with the models, controllers and routes covering everything needed to add, update and remove items. The team implemented the profile page, vendor order management and student order tracking all fully connected connected to the frontend and backend. Unit tests were written for the added controller functions.

The sprint was not without any setbacks. Reflecting on the sprint, the biggest lesson was on the importance of agreeing on interfaces before any development as the cart backend APIs were built but the frontend components were not connected to each other or the backend. Building components in isolation without shared understanding of how they could connect led to the cart not working. It would have saved a lot of time if testing code functionality before pushing was done. Teammates could have done better by doing the work assigned to them correctly and submitting in time. The team could have deployed the site in a different platform instead of continuing working on something that was failing.

Coming out of Sprint 2, the team identified several things to carry forward. The 4+1 model diagrams covering both Sprint 1 and Sprint 2 needed to be completed. Azure deployment still needed to be properly explored and set up. Unit testing and TDD practices that had been discussed during the sprint needed to be properly adopted going into Sprint 3. Bug fixes that had been worked on during the sprint also needed to be verified to make sure they were fully resolved and no longer affecting functionality.

Sprint Retrospective meeting 3:

Sprint 3 was marked by overcoming the team's biggest milestone which was getting the platform fully deployed. The admin order-view feature was implemented, with new backend endpoints added to fetch orders linked to specific student and order profiles and the fronted profile modals were updated to display information properly. The full review system was implemented, allowing students to submit reviews on collected order and vendors to view submitted reviews.

More time should have been dedicated to testing different scenarios and edge cases earlier in the development process rather than leaving through testing until the feature was mostly complete. The team also reflected that participation issues from other members should have been addressed properly as this caused setbacks on the development process. Continuing to do the responsibilities other members failed to do was difficult and took so much time that would have been used to implement more features.

Heading into Sprint 4, the team set out clear priorities. The focus would be on completing as many of the remaining user stories as possible so that the final phase of the project could be dedicated to polishing and refining the application before submission. With deployment now set up, all the attention could be focused on making sure everything works without problems. The team also made the decision to give out tasks to members who are willing to do the work. External user testing would be introduced to surface bugs that the development team had stopped noticing due to familiarity with the code. Members committed to tightening up testing practices and ensuring that what was submitted truly reflected the quality of the work that had gone into it.

Sprint Retrospective Meeting 4:

Sprint 4 was filled with a lot of feature implementation and finishing the remaining things required for the platform. The admin analytics dashboard was designed and built using React and Recharts. The dashboard has 5 graphs with the CSV export functionality which allows admins to download data for analysis. Menu item availability was implemented for both students and vendors, and allergen and dietary tagging was added in line with South African Regulation R146 and SANHA standards. The overview dashboards for both vendors and students were implemented. The full test suite for the sprint was written, and external user testing was conducted, which surfaced real-world bugs that the team had stopped seeing due to familiarity with the system. Deployment, component, and activity diagrams were all produced for the sprint.

The implementation of the CSV export feature was difficult which highlighted the importance of testing outputs int the environments where they would actually be used. Had the exported file been opened earlier in Excel, the problem would have been fixed sooner. More broadly, the participation issue should have been escalated and acted on in Sprint 1 or Sprint 2 rather than Sprint 4. Earlier action would have reduced the workload imbalance that the committed members carried throughout most of the project.

Going into the final stretch, the team's focus shifted to polishing the application, completing all documentation, and preparing everything for submission. Any problems where to be fixed and have a fully working platform. All architectural drawings, meeting minutes, and requirements engineering documentation were to be compiled into a single, complete set ready for submission. The team committed to closing off any outstanding tasks and doing a final review of the application to make sure everything was working as expected.